In [8]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
```

In [12]:
```
file_path = r"C:\Users\rable\OneDrive\Desktop\Raman DV\Raman dataset.xlsx"

df = pd.read_excel(file_path)

df.head()
```

Out[12]:

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view
0	2014-05-02	313000.0	3	1.50	1340	7912	1.5	0	0
1	2014-05-02	2384000.0	5	2.50	3650	9050	2.0	0	4
2	2014-05-02	342000.0	3	2.00	1930	11947	1.0	0	0
3	2014-05-02	420000.0	3	2.25	2000	8030	1.0	0	0
4	2014-05-02	550000.0	4	2.50	1940	10500	1.0	0	0

In [14]:
```
print("Shape:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nData Types:")
print(df.dtypes)
```

```
Shape: (4600, 18)

Columns:
Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'sqft_above',
       'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
       'statezip', 'country'],
      dtype='object')

Data Types:
date             datetime64[ns]
price                   float64
bedrooms                  int64
bathrooms               float64
sqft_living               int64
sqft_lot                  int64
floors                  float64
waterfront                int64
view                      int64
condition                 int64
sqft_above                int64
sqft_basement             int64
yr_built                  int64
yr_renovated              int64
street                   object
city                     object
statezip                 object
country                  object
dtype: object
```

In [16]:
```
df.describe()
```

Out[16]:

	date	price	bedrooms	bathrooms	sqft_living	sqft
count	4600	4.600000e+03	4600.000000	4600.000000	4600.000000	4.600000e
mean	2014-06-07 03:14:42.782608640	5.519630e+05	3.400870	2.160815	2139.346957	1.485252e
min	2014-05-02 00:00:00	0.000000e+00	0.000000	0.000000	370.000000	6.380000e
25%	2014-05-21 00:00:00	3.228750e+05	3.000000	1.750000	1460.000000	5.000750e
50%	2014-06-09 00:00:00	4.609435e+05	3.000000	2.250000	1980.000000	7.683000e
75%	2014-06-24 00:00:00	6.549625e+05	4.000000	2.500000	2620.000000	1.100125e
max	2014-07-10 00:00:00	2.659000e+07	9.000000	8.000000	13540.000000	1.074218e
std	NaN	5.638347e+05	0.908848	0.783781	963.206916	3.588444e

In [18]:
```
important_cols = ["price", "bedrooms", "bathrooms", "sqft_living", "sqft_lot"]

summary = pd.DataFrame({
    "Mean": df[important_cols].mean(),
    "Median": df[important_cols].median(),
    "Mode": df[important_cols].mode().iloc[0]
})

summary
```

Out[18]:

	Mean	Median	Mode
price	551962.988473	460943.461539	0.0
bedrooms	3.400870	3.000000	3.0
bathrooms	2.160815	2.250000	2.5
sqft_living	2139.346957	1980.000000	1720.0
sqft_lot	14852.516087	7683.000000	5000.0

In [20]:
```
df.isnull().sum()
```

Out[20]:
```
date             0
price            0
bedrooms         0
bathrooms        0
sqft_living      0
sqft_lot         0
floors           0
waterfront       0
view             0
condition        0
sqft_above       0
sqft_basement    0
yr_built         0
yr_renovated     0
street           0
city             0
statezip         0
country          0
dtype: int64
```

In [22]:
```
df.duplicated().sum()
```

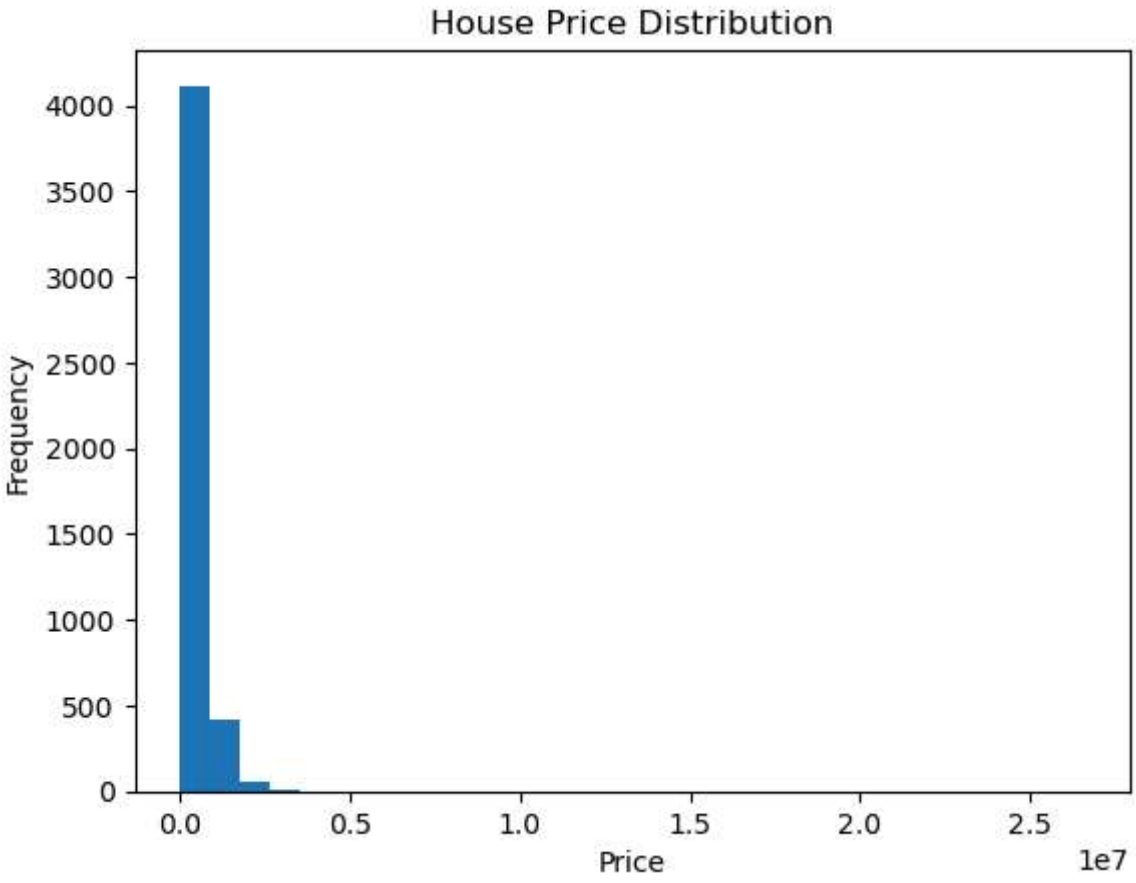
Out[22]:
```
0
```

In [24]:
```
df_clean = df.copy()

# Fill missing values
for col in df_clean.select_dtypes(include=["int64", "float64"]).columns:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

for col in df_clean.select_dtypes(include=["object"]).columns:
```

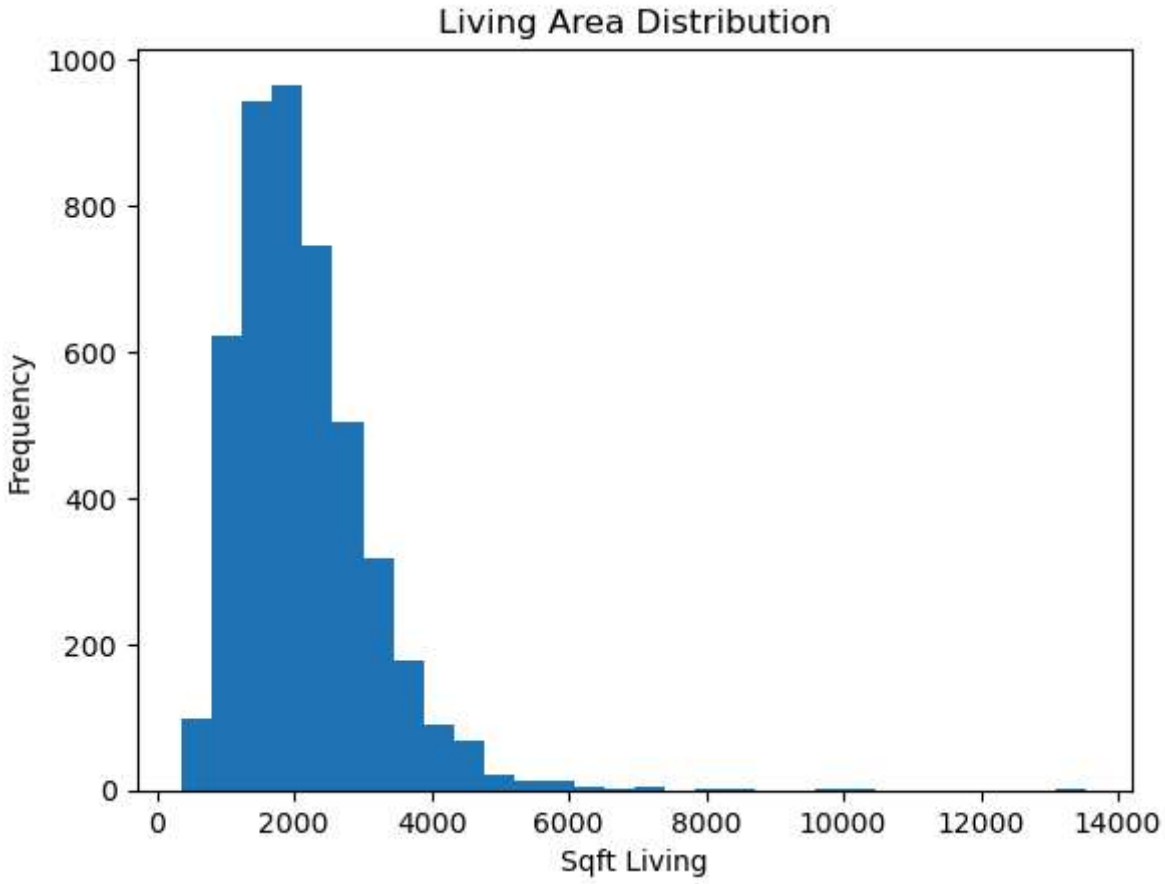
```
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

# Remove duplicates
df_clean = df_clean.drop_duplicates()

df_clean.shape
```

Out[24]: (4600, 18)

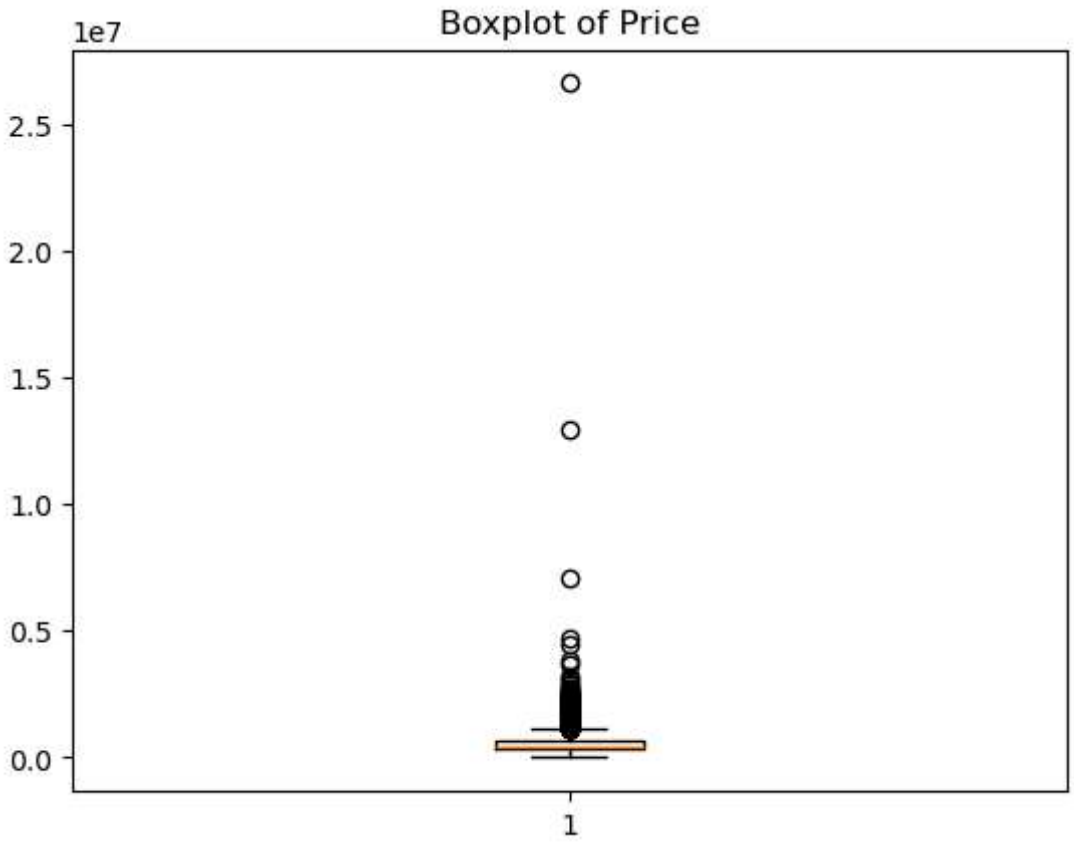
In [26]:
```
plt.hist(df_clean["price"], bins=30)
plt.title("House Price Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
```



In [28]:
```
plt.hist(df_clean["sqft_living"], bins=30)
plt.title("Living Area Distribution")
plt.xlabel("Sqft Living")
plt.ylabel("Frequency")
plt.show()
```



In [30]:
```
plt.boxplot(df_clean["price"])
plt.title("Boxplot of Price")
plt.show()
```

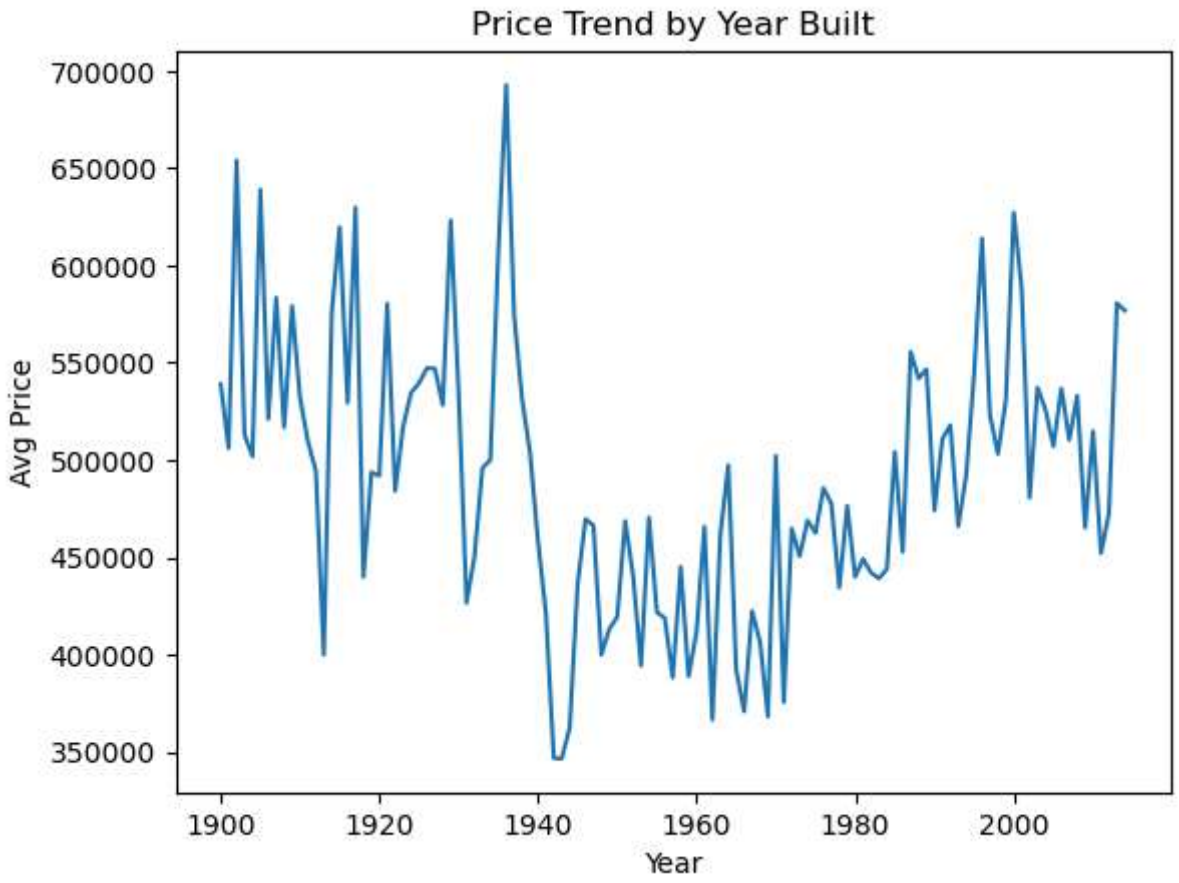


In [32]:
```
Q1 = df_clean["price"].quantile(0.25)
Q3 = df_clean["price"].quantile(0.75)
IQR = Q3 - Q1

df_no_outliers = df_clean[
    (df_clean["price"] >= Q1 - 1.5*IQR) &
    (df_clean["price"] <= Q3 + 1.5*IQR)
]

df_no_outliers.shape
```

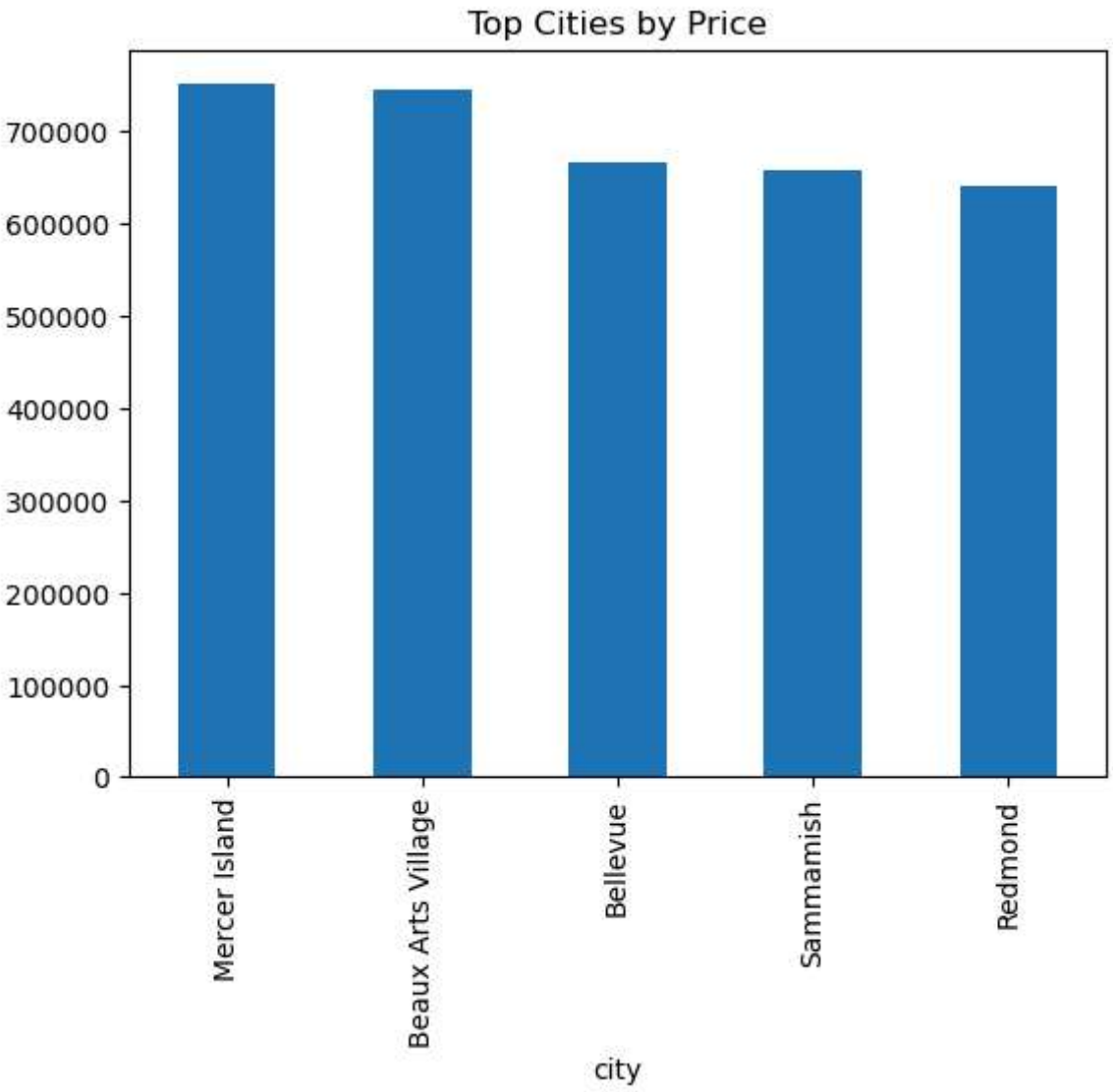
Out[32]: (4360, 18)

In [34]:
```
plt.scatter(df_no_outliers["sqft_living"], df_no_outliers["price"])
plt.title("Sqft Living vs Price")
plt.xlabel("Sqft Living")
plt.ylabel("Price")
plt.show()
```

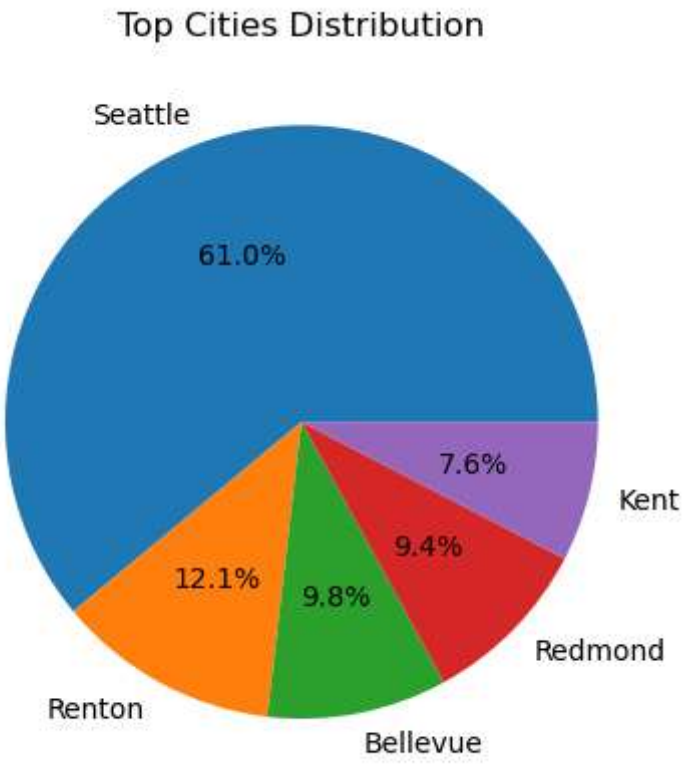
In [36]:
```
year_price = df_no_outliers.groupby("yr_built")["price"].mean()

plt.plot(year_price.index, year_price.values)
plt.title("Price Trend by Year Built")
plt.xlabel("Year")
plt.ylabel("Avg Price")
plt.show()
```



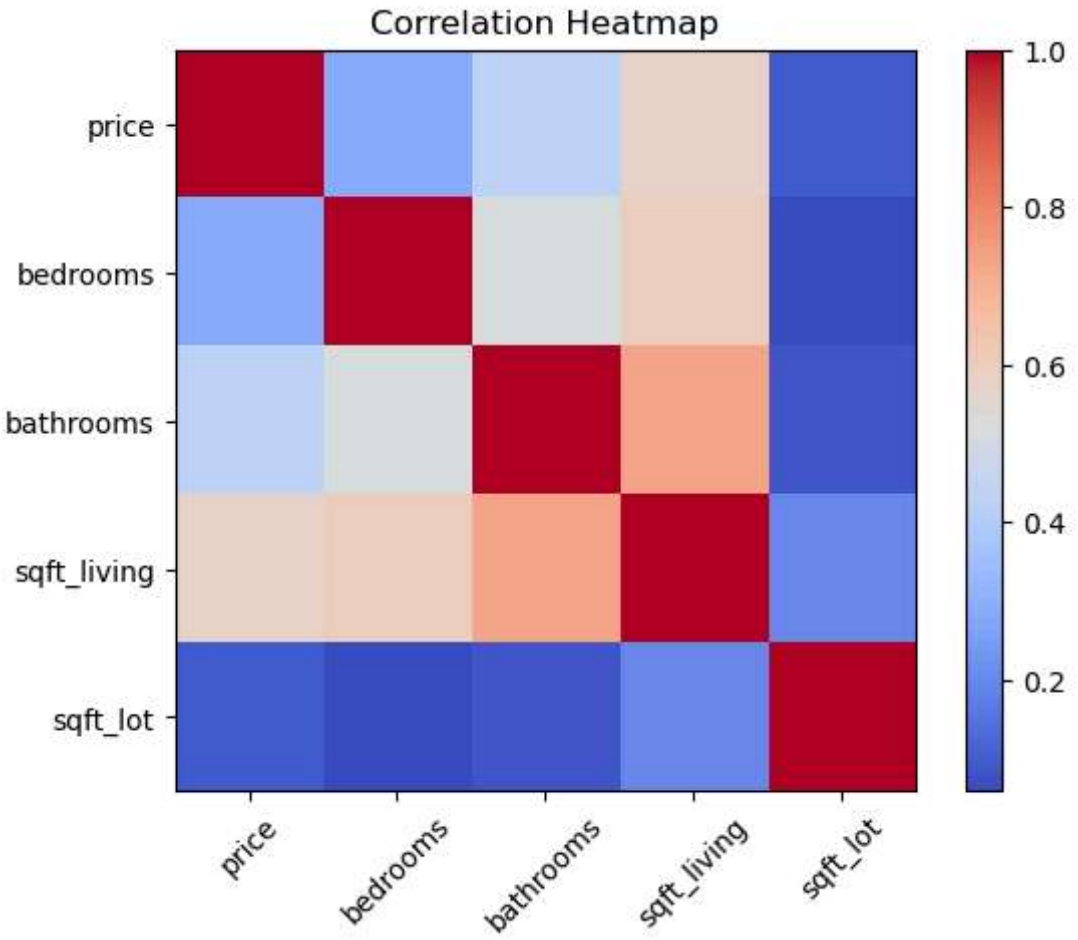
In [38]:
```
top_cities = df_no_outliers.groupby("city")["price"].mean().sort_values(ascending=F

top_cities.plot(kind="bar")
plt.title("Top Cities by Price")
plt.show()
```



In [40]:
```
counts = df_no_outliers["city"].value_counts().head(5)

plt.pie(counts, labels=counts.index, autopct="%1.1f%%")
plt.title("Top Cities Distribution")
plt.show()
```



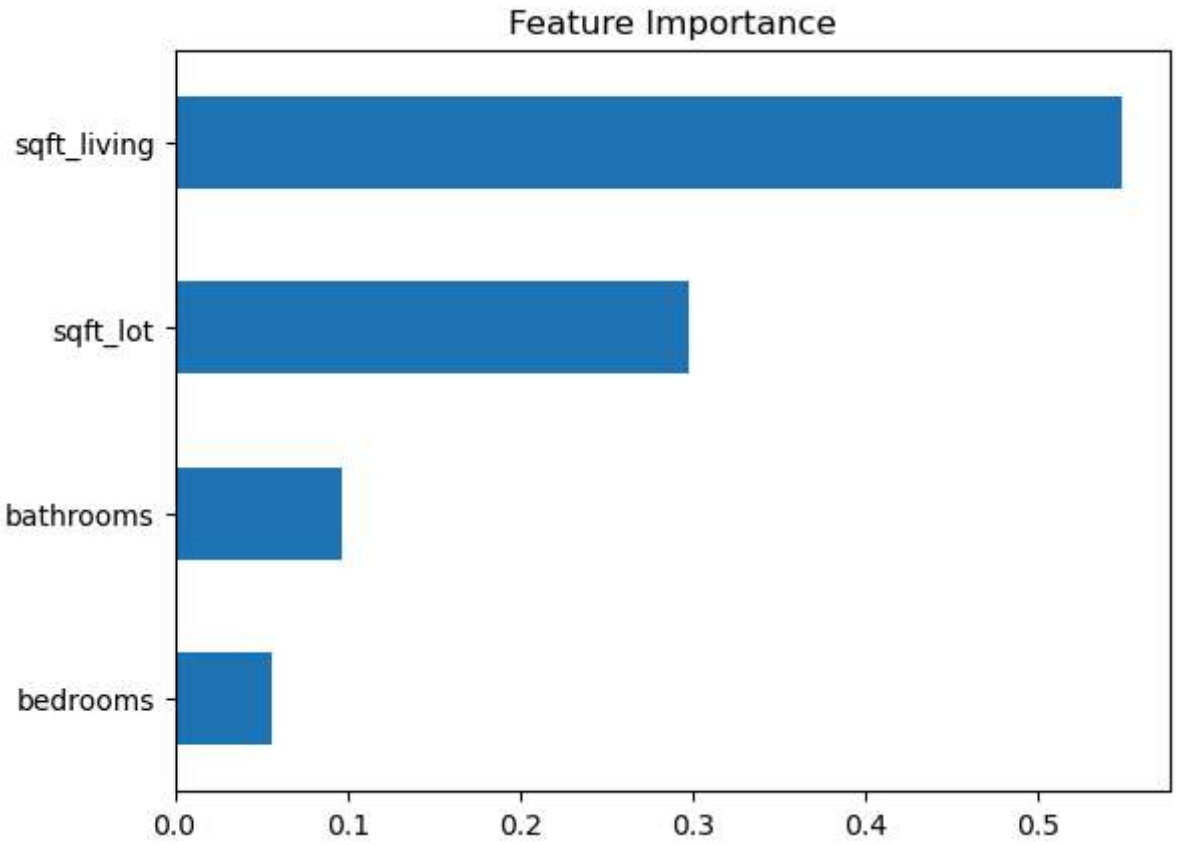
In [42]:
```
cols = ["price","bedrooms","bathrooms","sqft_living","sqft_lot"]

corr = df_no_outliers[cols].corr()

plt.imshow(corr, cmap="coolwarm")
plt.colorbar()

plt.xticks(range(len(cols)), cols, rotation=45)
plt.yticks(range(len(cols)), cols)

plt.title("Correlation Heatmap")
plt.show()
```



In [44]:
```
X = df_no_outliers[["bedrooms","bathrooms","sqft_living","sqft_lot"]]
y = df_no_outliers["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestRegressor()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("R2:", r2_score(y_test, pred))
```

```
MAE: 134336.38393239587
R2: 0.3123328001162421
```

In [46]:
```
importance = pd.Series(model.feature_importances_, index=X.columns)

importance.sort_values().plot(kind="barh")
plt.title("Feature Importance")
plt.show()
```



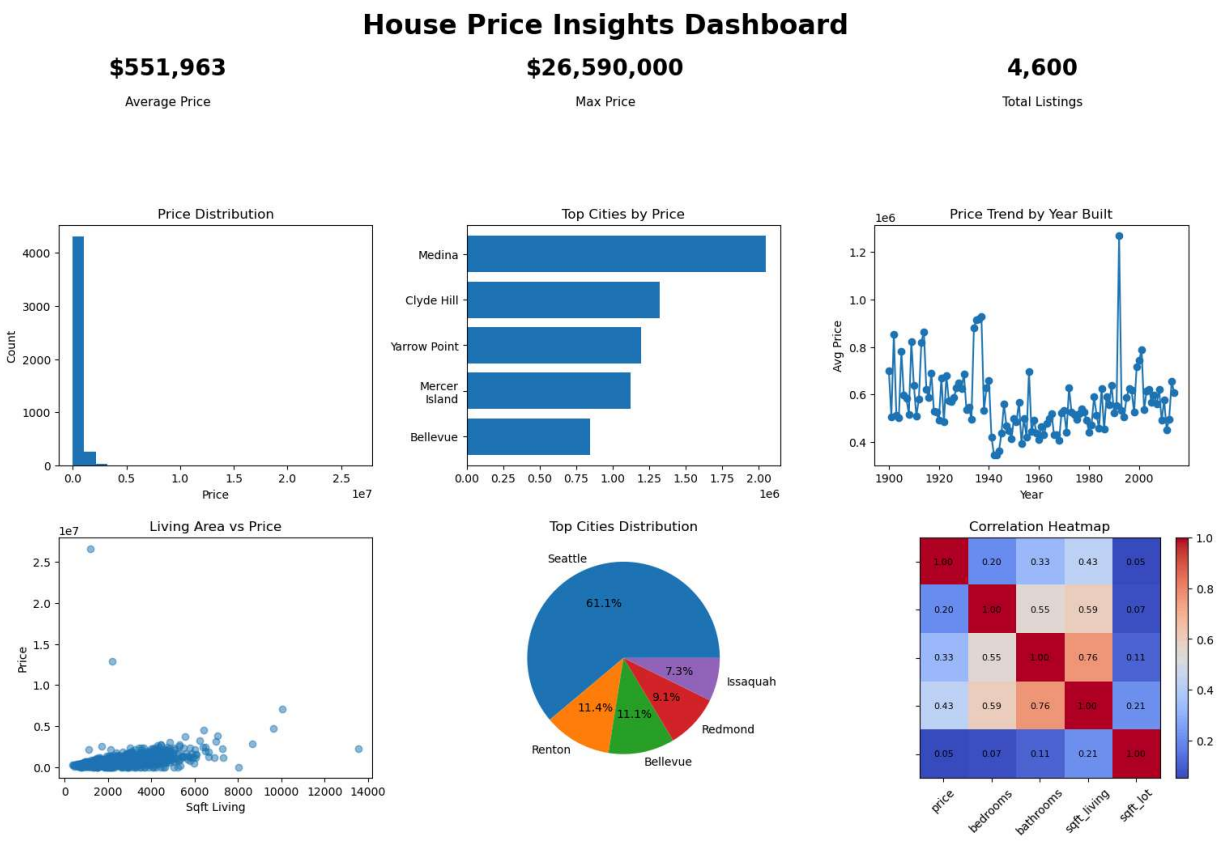
In [48]:
```
print("Average Price:", round(df_clean["price"].mean(),2))
print("Max Price:", df_clean["price"].max())
print("Total Houses:", df_clean.shape[0])
```

```
Average Price: 551962.99
Max Price: 26590000.0
Total Houses: 4600
```

In [52]:
```
import matplotlib.pyplot as plt
import textwrap

# ===== KPI CALCULATIONS =====
avg_price = df_clean["price"].mean()
max_price = df_clean["price"].max()
total_houses = df_clean.shape[0]

# ===== FIGURE LAYOUT =====
fig = plt.figure(figsize=(18, 11))
fig.patch.set_facecolor("white")

gs = fig.add_gridspec(3, 3, height_ratios=[0.5, 2, 2])

fig.suptitle("House Price Insights Dashboard",
             fontsize=24, fontweight="bold", y=0.97)

# ===== KPI SECTION =====
kpi = fig.add_subplot(gs[0, :])
kpi.axis("off")

fig.text(0.2, 0.90, f"${avg_price:,.0f}", fontsize=20, fontweight="bold", ha="cente
```

```
fig.text(0.2, 0.865, "Average Price", fontsize=11, ha="center")

fig.text(0.5, 0.90, f"${max_price:,.0f}", fontsize=20, fontweight="bold", ha="cente
fig.text(0.5, 0.865, "Max Price", fontsize=11, ha="center")

fig.text(0.8, 0.90, f"{total_houses:,}", fontsize=20, fontweight="bold", ha="center
fig.text(0.8, 0.865, "Total Listings", fontsize=11, ha="center")

# ===== CHART GRID =====
axes = [fig.add_subplot(gs[i, j]) for i in range(1, 3) for j in range(3)]

# ===== 1. Histogram =====
axes[0].hist(df_clean["price"], bins=25)
axes[0].set_title("Price Distribution")
axes[0].set_xlabel("Price")
axes[0].set_ylabel("Count")

# ===== 2. Top Cities =====
top_cities = df_clean.groupby("city")["price"].mean().sort_values(ascending=False).
wrapped = [textwrap.fill(city, 12) for city in top_cities.index]

axes[1].barh(wrapped, top_cities.values)
axes[1].set_title("Top Cities by Price")
axes[1].invert_yaxis()

# ===== 3. Trend =====
year_price = df_clean.groupby("yr_built")["price"].mean()

axes[2].plot(year_price.index, year_price.values, marker="o")
axes[2].set_title("Price Trend by Year Built")
axes[2].set_xlabel("Year")
axes[2].set_ylabel("Avg Price")

# ===== 4. Scatter =====
axes[3].scatter(df_clean["sqft_living"], df_clean["price"], alpha=0.5)
axes[3].set_title("Living Area vs Price")
axes[3].set_xlabel("Sqft Living")
axes[3].set_ylabel("Price")

# ===== 5. Pie =====
city_counts = df_clean["city"].value_counts().head(5)

axes[4].pie(city_counts, labels=city_counts.index, autopct="%1.1f%%")
axes[4].set_title("Top Cities Distribution")

# ===== 6. Heatmap =====
cols = ["price", "bedrooms", "bathrooms", "sqft_living", "sqft_lot"]
corr = df_clean[cols].corr()

im = axes[5].imshow(corr, cmap="coolwarm")

axes[5].set_xticks(range(len(cols)))
axes[5].set_yticks(range(len(cols)))
axes[5].set_xticklabels(cols, rotation=45)
axes[5].set_yticklabels([])
```

```
for i in range(len(cols)):
    for j in range(len(cols)):
        axes[5].text(j, i, f"{corr.iloc[i, j]:.2f}",
                     ha="center", va="center", fontsize=8)

axes[5].set_title("Correlation Heatmap")

fig.colorbar(im, ax=axes[5], fraction=0.04)

# ===== SPACING =====
plt.subplots_adjust(top=0.88, hspace=0.4, wspace=0.3)

plt.show()
```



In [ ]: